

A FREE PRACTICAL GUIDE · 2026 EDITION

The Brand Consultant's AI Content Dashboard

Build a local operating system in Claude Code that researches, plans, drafts and reviews brand content — while a human stays responsible for every claim that ships.

2-4 hrs

To a working skeleton you can demo to a client

5

Context files and five skills is the whole system

30 days

To a repeatable, billable engagement offer

READ IN 20 MINUTES

START HERE

The whole system in 60 seconds

If you read nothing else, read this page. Everything after it is detail, examples and the parts people get wrong.

1 • Own the context

Five markdown files describing the brand: brief, audience, voice, proof, red lines. These are assets. They move with you between tools and they outlive any one AI product.

2 • Turn repeat work into skills

Every time you re-explain a process, write it down once as a skill. Five skills cover the whole content cycle: brief, research, hooks, draft, review.

3 • Keep a human gate

Nothing publishes, sends, buys or deletes without a person approving it. Guardrails that matter are enforced in permissions and hooks, not written as polite instructions.

4 • Run a weekly rhythm

Brief on Monday, draft Tuesday, review Wednesday, approve Thursday, measure Friday. A dashboard that does not produce decisions on a cadence is a folder.

THE ONE-LINE VERSION

A content dashboard is not a visual dashboard. It is a source-of-truth workspace that tells an AI collaborator who the brand is, what is in flight, which evidence supports a claim, and what "ready to publish" means.

What is in this guide

01	Chat vs. agents, and what the consultant still owns	Why the shift is real and where it is oversold
02	Build the workspace	Folder structure, five context files, calendar schema
03	Pick the right steering primitive	The decision table most guides get wrong
04	The five skills to install	Working frontmatter you can copy
05	Make the dashboard live	Injected commands, subagents, scheduling
06	Tools, MCP and the human check on each	A deliberately small stack
07	Guardrails that actually hold	Permissions, hooks, prompt injection, least privilege
08	The consultant playbook	Discovery, packaging, handoff, 30-day plan

USE THIS RESPONSIBLY

AI accelerates research and production. It also invents facts, flattens a voice, and mishandles private information. Treat every output as a draft. Keep credentials out of prompts and repositories. Never auto-publish or send client communications without a human approval step.

Chat vs. agents, and what you still own

The distinction is not convenience. It is whether work actually leaves your plate.

DIMENSION	CHAT	AGENT
Input	A question	A goal
Output	An answer you then act on	A finished artifact
Who works	You	The agent, looping until done
Best for	Lookups, quick decisions, sanity checks	Multi-step production work with defined outputs
Failure mode	You become the bottleneck	Confident, well-formatted, wrong

The loop underneath

Every agent harness runs the same three steps until a goal is met: **observe** the current state, **think** about the single best next action, **act**, then loop. A single task can run twenty or thirty loops. The agent stops when it judges the output matches the goal you defined.

Which means the highest-leverage thing you write is the definition of *done*. "Research competitors" has no finish line. "Read the ten most-engaged LinkedIn posts from these four accounts, tag the hook mechanism on each, and write a cited pattern memo to `research/briefs/`" has three.

HARNESSES, NOT AGENTS

Claude Code, Claude Cowork, Codex and similar tools are all interfaces to the same loop. When someone says they built an agent, they almost always mean they configured one that already existed. That is good news: the work is context and process design, not engineering.

What the consultant owns

This is the part that determines whether you are worth hiring. The system can prepare, organize, compare, draft and surface decisions. It cannot own any of the following.

- **Truth.** Which claims the brand can actually make, and what evidence backs each one.
- **Strategy.** Which audience matters this quarter and what decision you want them to make.
- **Legal and regulatory exposure.** Health, finance, income and efficacy claims are yours.
- **Relationships.** Testimonial permissions, partner sensitivities, what a client will hate.
- **Taste.** An AI will produce competent, average, on-brand-adjacent output forever unless a person keeps raising the floor.

THE HONEST CAVEAT

Productivity claims in this space run from 5x to 10x. Treat them as ceiling numbers from operators with mature systems, not as what you get in week one. The realistic first win is smaller and more useful: research and first drafts stop being the slow part, and your time moves to judgment, evidence and distribution.

Build the workspace

One folder per brand. Simple enough to navigate without an app, structured enough that an agent never has to guess.

```
brand-content-os/
├── CLAUDE.md           # operating contract – loads every session
├── README.md           # how a human uses this folder
├── context/            # stable truth: brand-brief, audience, voice,
                        #   offer-and-proof, red-lines
├── knowledge/          # evidence: sources/, customer-language/,
                        #   competitor-notes/, content-learnings.md
├── dashboard/          # live queue: content-calendar, content-backlog,
                        #   metrics-weekly
├── research/           # inbox/ and briefs/
├── drafts/             # in-progress/ → review/ → approved/
├── published/          # post-log.md
└── .claude/            # skills/ agents/ rules/ settings.json
```

The five context files

FILE	WHAT GOES IN IT
brand-brief.md	Mission, category, business model, price range, the outcome the audience buys, named competitors, current 90-day priority.
audience.md	Segments, jobs to be done, objections in the customer's own words, awareness level, where they actually consume content.
voice.md	Five to ten <i>observable</i> rules with examples. Sentence length, words to use and avoid, point of view, humor level, how strong a claim may be. "Confident but warm" is not a rule.
offer-and-proof.md	What is sold, who it helps, documented proof with a source note for each item, precise limits, approved testimonials and their permission status.
red-lines.md	Claims the brand must never make, topics it will not touch, competitor references it will not use, regulated language.

READ EVERY LINE BEFORE YOU DEPLOY

One wrong sentence in `audience.md` or `offer-and-proof.md` poisons every downstream draft, quietly and consistently. These files are the highest-leverage editing you will do all engagement. Date them and assign an owner.

A dashboard row that is legible to both people and AI

Each proposed piece in `dashboard/content-calendar.md` carries: ID · working title · pillar · target segment · platform · objective · status · owner · source links · proof required · CTA · deadline · approval field. Statuses stay fixed: *idea* → *brief* → *drafting* → *review* → *approved* → *published* → *measured*. A fixed status set is what makes the file queryable by an agent, and what lets a client see the pipeline without asking you.

Pick the right steering primitive

This is the table most guides skip, and skipping it is why people's setups get slow, expensive and unreliable at month two.

Claude Code offers several ways to instruct behavior. They differ on three axes: **when the instruction loads**, **whether it survives a long session**, and **how much authority it carries**. Putting everything in one master file is the single most common mistake.

METHOD	WHEN IT LOADS	CONTEXT COST	USE IT FOR
CLAUDE.md (root)	Session start; stays all session	High — every line costs whether relevant or not	Always-needed facts: file naming, where drafts live, hard working conventions
CLAUDE.md (subfolder)	On demand, when a file in that folder is read	Low	Conventions for one area, e.g. rules for files inside <code>drafts/review/</code>
Rules <code>.CLAUDE/RULES/</code>	Session start, or only on matching files if you add a <code>paths:</code> field	Medium; low when path-scoped	Cross-cutting constraints, e.g. every research brief must cite a source filename
Skills <code>.CLAUDE/SKILLS/</code>	Name and description at start; full body only when invoked	Low	Procedures: brief, research sprint, hook lab, draft, brand review
Subagents <code>.CLAUDE/AGENTS/</code>	Never enters main context; runs in its own window	Low	Side work whose intermediate output you will never re-read: competitor sweeps, log analysis
Hooks <code>SETTINGS.JSON</code>	Fires on lifecycle events; bypasses compaction entirely	Low	Deterministic automation and real blocking: run a check after every edit, deny a command
Output styles	Session start, injected into the system prompt	High, and it overwrites defaults	Major role changes only. Use the built-in styles first.

Load behavior, compaction and context cost per Anthropic, "Steering Claude Code: when to use CLAUDE.md, skills, hooks, and subagents," 18 June 2026.

Four signals you have put something in the wrong place

IF YOU WROTE THIS...	MOVE IT HERE
"Every time X, always do Y" in CLAUDE.md	A hook . The model choosing to run a check is not the same as the check running.
"Never do this" in CLAUDE.md	Permissions or a hook . An instruction is followed most of the time. Under a long session, an ambiguous case, or content that contains injected instructions, it can fail. A real guardrail is deterministic.
A 30-line procedure in CLAUDE.md	A skill , where the body loads only on use.
A rule that only applies to one folder	A path-scoped rule . An unscoped rule is mechanically identical to pasting it into CLAUDE.md.

TWO MYTHS WORTH KILLING

Splitting CLAUDE.md into @ imports does not save context. Imported files load at launch exactly like the parent. It buys organization, not budget. Path-scoped rules and skills are what actually reduce load.

More instructions do not mean better adherence. Adding "always double-check your work" to current-generation models produces over-verification: work that was already correct gets re-checked, burning turns. Keep CLAUDE.md under 200 lines, give it an owner, and review edits to it the way you would review a contract.

Starter CLAUDE.md

```
# Brand Content OS

## Mission
Produce evidence-led content for [BRAND]. Optimize for qualified trust, not volume.

## Read before touching a draft
- context/brand-brief.md
- context/audience.md
- context/voice.md
- context/offer-and-proof.md
- context/red-lines.md

## Operating rules
- Files under knowledge/sources/ are evidence. Cite their filenames in every brief.
- Never invent customer stories, results, statistics, product capabilities, or
  legal, medical or financial claims.
- Label unsupported assertions as open questions, or remove them.
- Draft into drafts/in-progress/ unless told otherwise.
- Do not publish, send, upload, delete or change any external account.
- Every draft ends with: audience, promise, evidence used, CTA, and a
  self-review against voice.md.

## File conventions
Name files YYYY-MM-DD--short-slug.md. Never overwrite approved work;
create a new revision.
```

The five skills to install

A skill is a folder containing a `SKILL.md`. Its description tells Claude when to reach for it; its body is the procedure. This is where your intellectual property lives.

01 content-brief

In: a topic or campaign. **Out:** a decision-ready brief with objective, audience, three angles, approved evidence, evidence gaps, recommended CTA, and the one decision needed from you. It does not draft.

02 research-sprint

In: a question and a scope. **Out:** a dated memo in `research/briefs/` with source URLs and filenames, exact claims, counterpoints, and an explicit "not verified" list. Run it in an isolated subagent so the raw scraping never enters your main thread.

03 hook-lab

In: an approved brief. **Out:** fifteen platform-specific hooks, each labelled by mechanism — specificity, tension, contrarian, aspiration, consequence, proof. You pick the winner. The model does not get to declare one by confidence alone.

04 draft-content

In: an approved brief and a chosen hook. **Out:** a platform draft using only approved proof, plus asset notes, evidence list and stated assumptions.

05 brand-review

In: a draft. **Out:** a pass/fail checklist across factual support, voice, audience fit, disclosure, CTA clarity, platform fit and prohibited claims. It recommends edits. It never marks anything approved.

Working example: the content-brief skill

Save at `.claude/skills/content-brief/SKILL.md`.

```
---
name: content-brief
description: Create an evidence-led content brief for a brand topic. Use when
  planning a post, video, newsletter or campaign.
disable-model-invocation: true
argument-hint: "[topic or campaign]"
---
```

1. Read context/brand-brief.md, audience.md, offer-and-proof.md, red-lines.md.
2. Search knowledge/ and research/ for evidence related to \$ARGUMENTS.
3. Write research/briefs/YYYY-MM-DD--[slug].md containing:
 - objective and target segment
 - 3 angles, each with a tension and a promise
 - approved evidence, with source filenames or URLs
 - evidence gaps and open questions for the consultant
 - recommended CTA and per-platform adaptations
4. Do not write a full draft.
5. End with the single decision you need from the consultant.

The frontmatter fields that matter

FIELD	WHY A CONSULTANT CARES
description	How the skill gets found. Put the primary use case first — the listing text is truncated, and a buried trigger phrase means the skill never fires.
disable-model-invocation	Only you can run it. Put this on anything with a side effect: publishing, sending, spending.
context: fork	Runs the skill in an isolated subagent. Ideal for research sprints; only the summary comes back.
allowed-tools	Pre-approves specific tools for that turn so you are not clicking through prompts mid-workflow.
paths	Auto-loads only when working with matching files.
argument-hint	Shows expected input in autocomplete. Small thing; makes a client handoff feel finished.

TWO CONSTRAINTS TO DESIGN AROUND

Keep each `SKILL.md` under roughly 500 lines and push long reference material into sibling files the skill points to. And remember that once a skill is invoked its content stays in context across the rest of the conversation, so every line is a recurring cost. State what to do; skip the narration.

PORTABILITY NOTE

If you plan to hand a client skills they will use outside Claude Code — uploaded to a Claude account or through the API — only a small set of frontmatter fields is accepted there: name, description, license, compatibility, metadata, allowed-tools. Extra fields cause a hard error on upload rather than being ignored. Build client-facing skills to that narrower spec.

Make the dashboard live

Three moves that turn a folder of files into something that reports its own status.

1 • Inject live state into a skill

A skill body can run shell commands before Claude ever sees it, with the output inlined. This is how you get a **standup** that reflects reality rather than a summary of what Claude remembers.

```
---
name: content-standup
description: Morning status across the content pipeline
allowed-tools: Bash(ls *) Bash(find *)
---

## Drafts in flight
!`ls -lt drafts/in-progress/ drafts/review/`

## Research collected this week
!`find research/briefs/ -mtime -7 -type f`

Summarize where each piece stands, flag anything stalled more than five
days, and list the decisions I need to make today.
```

A failed command aborts the whole skill invocation, so append `|| true` to anything that may exit non-zero.

2 • Push noisy work into a subagent

Competitor sweeps generate a lot of intermediate output you will never re-read. A subagent runs in its own context window and returns only its final message to your session, and they can nest several levels deep — which is how one "analyze these twelve accounts" instruction fans out without flooding your thread. Rule of thumb: reach for a **subagent** when you want a result, and a **skill** when you want to watch and steer each step.

3 • Schedule the recurring pieces

Once the manual weekly cycle is reliable, automate the boring end of it — a Friday metrics summary, a Monday backlog sweep.

THE GOTCHA THAT WASTES AN AFTERNOON

Scheduled work that runs as a fresh remote session does not read the skills folder on your machine. If a skill exists only in your personal local folder, the scheduled run reports it as not found. Either enable the skill for the account that runs the session, or commit it into the project's own `.claude/skills/` so it travels with the workspace.

4 • Test that your skills actually work

A skill firing tells you Claude found it, not that it did what you meant. Anthropic publishes a `skill-creator` plugin that runs each test prompt with and without the skill in isolated sessions, grades the outputs, reports the pass-rate improvement against the token overhead, and flags descriptions that trigger on the wrong requests. Worth running once you pass five or six skills, because that is when they start colliding.

Install it with `/plugin marketplace add anthropics/claude-plugins-official`, then `/plugin install skill-creator@claude-plugins-official`.

Tools, MCP, and the human check on each

Keep the stack small on purpose. Add a tool only when it produces a known input or removes a repeated bottleneck.

MCP (Model Context Protocol) is the standard that lets an agent connect to outside services as tools. It separates a system that advises on your marketing from one that runs parts of it — and it is where most of your risk lives. Scope every connection to the minimum permission that does the job.

NEED	TOOL CATEGORY	BEST USE	YOUR CHECK
Brand thinking	Claude Code plus local files	Read context, structure drafts, manage workflow artifacts	Is it true, distinctive and useful?
Web evidence	Search and primary sources	Current facts, studies, official product documentation	Open the source. Verify date, scope and exact wording.
Site capture	Web-to-markdown, e.g. Firecrawl	Turn permitted public pages into a readable research record	Respect access rules; check extraction against the original
Platform research	Platform scrapers, e.g. Apify	Study public patterns at volume where terms and law permit	No private data, no deceptive collection, no copied content
Integrations	Connector platforms, e.g. Composio	One connection exposing many apps without hand-rolling OAuth	Least privilege. Review every write action.
Browser tasks	Playwright, browser dev tools	Testing and controlled navigation	Never grant credentials casually. Never auto-post.
Email operations	Email platform with MCP support	Build and schedule campaigns from inside the workflow	Human approval before any send. Always.
Creative assets	Generative image and video tools	Concept and variation at volume	Rights, likeness, disclosure. Use official connectors only.
Input speed	Voice dictation	Speak long briefs instead of typing them	Proofread transcriptions before they become context files

SEQUENCING ADVICE

Connect nothing in week one. Local files and web search will carry your first three real pieces. Add a connector only after a manual step has annoyed you twice. Tool sprawl is the most common reason these systems get abandoned: every connector adds surface area to review, permissions to manage, and noise in the agent's tool list.

Where this leads

The fully connected end state: the agent reads a campaign brief, pulls live product data, applies the brand guidelines, generates creative variations, builds the email flow, stages the ad set, then **stops at a human approval gate**. The value is not any single step. It is removing the switching cost between them.

Guardrails that actually hold

The difference between a rule you wrote and a rule that is enforced.

THE CORE PRINCIPLE

An instruction in a markdown file is followed most of the time. When something absolutely must not happen, that is not good enough. Deterministic enforcement lives in permissions and hooks — configuration outside the context window that the model cannot talk itself past.

Non-negotiables

- **Least privilege.** A research task should not hold permission to publish, send email, or touch payment systems. Split capabilities across separate scopes rather than granting one agent everything.
- **Secrets stay out.** API keys, passwords, client personal data and unredacted sensitive documents never go into prompts or tracked files. Use environment variables and access controls.
- **Treat retrieved text as untrusted.** Web pages, PDFs, social posts and documents can contain text written to manipulate an agent that reads them. Content pulled from the internet is data, not instructions. If a scraped page appears to be telling your agent to do something, that is the signal to stop and look.
- **Human gate on irreversible actions.** Posts, external messages, ads, regulated claims, invoices, purchases, account changes, deletions. All of them.
- **Preserve citations.** Keep three categories separate in every research brief: *observed pattern*, *client claim*, and *verified evidence*. Conflating them is how a brand ends up defending a statistic nobody can source.
- **Version approved work.** Never let a workflow overwrite an approved file. New revision, new filename, old one intact.

A short blocking pattern

Hooks fire on lifecycle events — before a tool call, after a file edit, at session start. A pre-tool-use hook can inspect a call and exit with a failure code to deny it outright. That is the mechanism for "this workspace can never run a publish command," and it works regardless of how the conversation got there.

Alongside hooks, the permissions system lets you allow or deny specific tools and specific skills by name. Combining both gives you a workspace you can hand to a junior team member, or a client, without worrying about what a long session might talk itself into.

DISCLOSURE

Decide your position on AI disclosure with the client before the first post ships, not after someone asks. Platform rules differ, some jurisdictions have advertising disclosure requirements, and "we'll figure it out later" is a bad answer to give in public.

The consultant playbook

The value is not having AI. It is translating a brand into decisions, evidence and a process that survives past one clever prompt.

The weekly rhythm

DAY	YOUR DECISION	SYSTEM TASK	ARTIFACT
Monday	Choose 3–5 priority topics	<code>/content-brief</code> on each	Cited briefs in <code>research/briefs/</code>
Tuesday	Approve angles and proof	<code>/hook-lab</code> then <code>/draft-content</code>	Drafts in <code>drafts/in-progress/</code>
Wednesday	Review claims, nuance, voice	<code>/brand-review</code> , then revise	Items moved to <code>drafts/review/</code>
Thursday	Final publish decision	Prepare captions, assets, post log	Approved package, published by hand
Friday	Decide what to repeat	Summarize metrics and learnings	<code>dashboard/metrics-weekly.md</code>

Discovery questions for the client

- What business result should content influence in the next 90 days?
- Which audience segment is most valuable right now, and what decision are they trying to make?
- What can the brand prove today? What must it never claim?
- What content has already produced qualified conversations or sales, and why do you think it worked?
- Which competitors are worth studying, and which should we deliberately not resemble?
- Who owns final approval, and what is the maximum acceptable risk of one incorrect post?

What to measure

Leading

Saves, shares, profile visits, replies, qualified DMs, email sign-ups, and time-to-produce.

Business

Discovery calls, conversions, repeat purchases, revenue attributable to a campaign, objections surfaced.

System health — the one most people skip

Percentage of drafts needing major correction, unsupported claims caught in review, workflow duration, and which skills are actually being reused. A skill nobody invokes twice was a bad skill.

Packaging the engagement

Three shapes that work, in rough order of how most consultants grow into them:

SHAPE	WHAT YOU DELIVER	WHAT TO WATCH
-------	------------------	---------------

Build sprint	A fixed-scope engagement: workspace, five context files, five skills, three worked examples, a handoff session	Scope creep into content production. Sell the system, not unlimited drafts.
Build plus retainer	The sprint, then a monthly cycle where you run the rhythm and keep the context current	Becoming a content agency with extra steps. Price the judgment, not the volume.
Enablement	You build it and train the client's team to operate it, with a review cadence	Highest margin, hardest sale. Needs a client who genuinely wants ownership.

Client handoff checklist

- A complete workspace folder with a readable README and CLAUDE.md
- Five current context files, each dated and assigned an owner
- A content calendar with the fixed status set in use
- Five documented skills and at least three completed real examples
- A list of approved integrations, permission scopes, account owners, and explicit no-go actions
- A weekly review ritual and a change log for edits to brand context or skill instructions

The 30-day build

WEEK	DO THIS
Week 1	Folder structure, five context files, a content backlog. Produce one brief manually, start to finish.
Week 2	Add content-brief, hook-lab and draft-content. Run them on three real pieces. Write down every correction you had to make.
Week 3	Add research-sprint and brand-review. Set a source policy. Lock the approval status field in the calendar.
Week 4	Review metrics, delete instructions that did nothing, tighten voice examples with real approved copy, and automate exactly one well-tested repetitive step.

FINAL PERSPECTIVE

Treat the system as a junior operating team. It can prepare, organize, compare, draft and surface decisions. It cannot own truth, strategy, legal exposure, relationships or brand judgment. The compounding advantage comes from keeping context accurate, turning repeat work into tested skills, and using every publishing cycle to improve the next one.

Sources and how this was verified

So you can check anything here yourself, and so you know what is framing versus what is documented behavior.

Framing

Open Residency, Ep. 037 — "AI Agents: The Most Valuable Skill You Can Learn in 2026," featuring Remy Gaskell. Published 22 July 2026, 2h24m. Source of the agent-OS concept, the observe-think-act framing, the context-tools-skills onboarding model, and the folder-as-operating-system idea. A companion playbook PDF is available from the episode page at openresidency.com.

Implementation

Anthropic, "Steering Claude Code: when to use CLAUDE.md, skills, hooks, and subagents," 18 June 2026. Source of the load-timing, compaction and context-cost comparison in Chapter 3, and of the guidance on when an instruction belongs in a hook or in permissions rather than a markdown file.

Claude Code documentation — skills, memory, subagents, hooks, permissions. code.claude.com/docs. Source of the frontmatter reference, skill discovery and precedence rules, the frontmatter portability limits, dynamic context injection syntax, and the behavior of scheduled and remote sessions.

Agent Skills open standard. agentskills.io. The specification that governs which frontmatter fields are portable outside Claude Code.

SHELF LIFE

The concepts in Chapters 1, 2, 7 and 8 are durable. The specific mechanics in Chapters 3, 4 and 5 move quickly — field names, defaults and version-gated behavior have all changed more than once this year. Before you build something a client depends on, check the current documentation for the specific fields you are relying on. This guide was verified in September 2026.

Tool mentions

Tools named in Chapter 6 are illustrative categories, not endorsements, and no provider named is required to build this system. Nothing in this guide is a commercial relationship. Verify pricing, terms of service and data handling for any tool before putting client work through it — particularly for anything that scrapes a platform or processes personal data.

Free to share. Pass this to anyone building a content practice. If you improve it, the version that matters is the one with your own worked examples in it — a guide with three real briefs attached beats a perfect template every time.